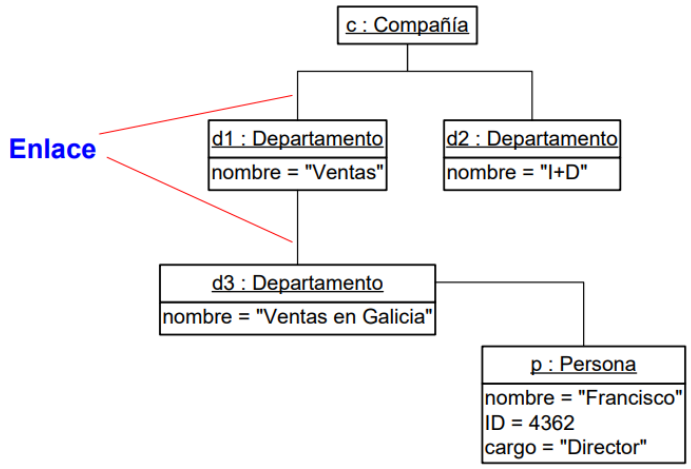
T6. Otros diagramas de UML

Diagrama de objetos

Modelan **instancias** de elementos contenidos en diagramas de clases. Su propósito es mostrar los **objetos** y sus relaciones en un momento concreto (**enlaces**). Proporcionan una **foto** estática de los objetos (instancias de clases) del sistema en un momento concreto del tiempo. Se usan para modelar **estructuras** de **datos**.

Puede incluir objetos de clases abstractas o atributos incompletos (porque es una especificación, no una ejecución real).



Diagramas de Paquetes

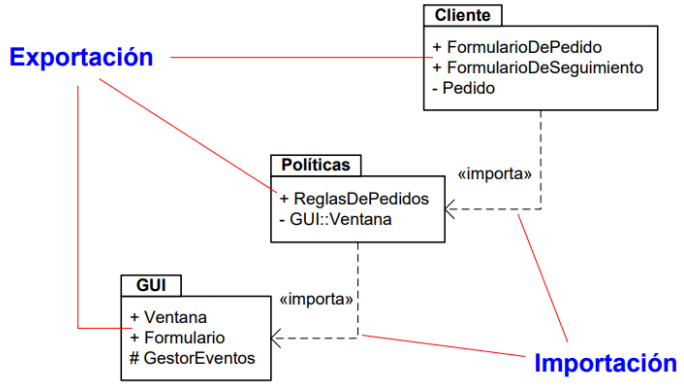
Un **paquete** es un mecanismo de propósito general para agrupar elementos de modelado, se pueden emplear para modelar las diferentes **vistas** de la arquitectura. Un paquete debe **agrupar elementos** cercanos **semánticamente** y que cambien juntos; puede contener clases, interfaces, componentes, nodos, colaboraciones, casos de uso, diagramas, otros paquetes...

Nota: cada elemento pertenece a un **solo paquete**.

Un paquete forma un **espacio de nombres**, de modo que los elementos del mismo tipo deben tener nombres únicos dentro de su paquete. No tienen identidad, por lo que no se instancian (invisibles en tiempo de ejecución).

El propósito de este diagrama es agrupar clases, interfaces y otros elementos, además de mostrar paquetes y **dependencias** entre ellos (lo que ayuda a gestionar la complejidad). Los paquetes pueden **contener otros** paquetes (descomposición jerárquica), existe un **paquete raíz** que abarca todo el sistema. Las dependencias indican que algún elemento interno depende de otro (no todos) y permite controlar visibilidad y encapsulamiento.

Nota: Un paquete depende de otro si existe una **dependencia entre** dos **elementos** cualesquiera de esos dos paquetes.



- **Importación**: Un paquete importa a otro cuando puede usar sus elementos. Se representa con una flecha discontinua y el estereotipo «importa». El paquete que importa depende del otro y puede usar sus elementos como si fuesen propios.

- **Exportación**: Un paquete exporta cuando hace visibles sus elementos para otros paquetes y no se dibuja explicitamente (exporta automaticamente cuando sus elementos son públicos, +, y cuando otro paquete hace «importa» hacia el).

El diagrama representa la organización de un sistema dividido en tres paquetes principales: **GUI**, **Políticas** y **Cliente**, junto con las relaciones de dependencia entre ellos.

- El **paquete GUI** actúa como la capa de interfaz de usuario. En él se encuentran elementos como *Ventana*, *Formulario* y *GestorEventos*, que son responsables de la interacción directa con el usuario. Este paquete **exporta** sus componentes para que puedan ser utilizados por otros paquetes.

- El **paquete Políticas**, tiene como principal elemento *ReglasDePedidos*. Sin embargo, en el diagrama se observa que este paquete **importa** elementos del paquete GUI, concretamente *GUI::Ventana*. Esto indica que **depende** directamente de la interfaz de usuario.

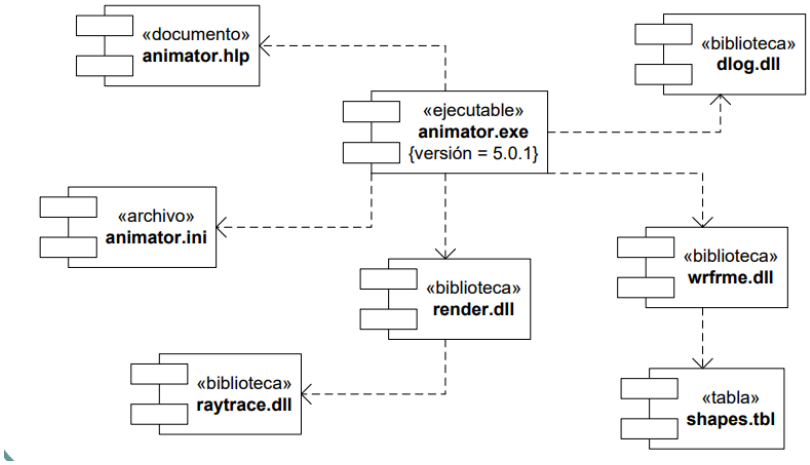
- El **paquete Cliente** incluye elementos como *FormularioDePedido*, *FormularioDeSeguimiento* y *Pedido*. **Importa** el paquete Políticas, Cliente depende de Políticas para funcionar correctamente.

Nota: Aunque en la foto aparezca que Cliente exporta no es así, hay un error. Que tenga elementos visibles (públicos) no implica que ofrezca esos elementos a otros, para que exporte otro paquete debe usar esos elementos (que importe al paquete).

Diagrama de Componentes

Los **componentes** son partes **reemplazables** de un sistema que soportan un conjunto de interfaces. Suponen **encapsulamiento físico** de clases e interfaces, principalmente. Modelan todo tipo de elementos físicos: ejecutables, librerías, archivos... Hay que tener en cuenta que una clase puede implementarse en **múltiples componentes**, pero se define en un **solo paquete**.

Este diagrama representa los componentes físicos de un sistema de software y sus dependencias. Su propósito es mostrar cómo el sistema está **dividido** en **módulos reutilizables** y refleja los elementos que pueden ser compilados, desplegados o reemplazados. Cada componente tiene nombre y puede tener estereotipos como <<executable>>, <<library>>, <<document>>...



En el centro del diagrama se encuentra el componente principal, identificado con el estereotipo **«ejecutable»**, denominado *animator.exe*, incluye una propiedad que indica su versión (*{versión = 5.0.1}*). Este componente actúa como el **núcleo** del sistema, ya que es el programa que se ejecuta y del que dependen el resto de elementos.

Las **relaciones entre componentes** se representan mediante **flechas discontinuas**, que indican dependencias. Estas flechas señalan que un componente necesita a otro, es decir, que utiliza sus servicios, datos o funcionalidades. La dirección de la flecha es clave: parte del componente que depende y apunta hacia el componente del que depende. En este caso, *animator.exe* depende de varios elementos, como bibliotecas, archivos y documentos.

Los distintos componentes del sistema están clasificados mediante **estereotipos** que permiten identificar la naturaleza de cada elemento. Así, aparecen componentes de tipo **«biblioteca»** (como *render.dll*, *raytrace.dll*, *dlog.dll* y *wrfme.dll*), que representan **módulos reutilizables** que proporcionan

funcionalidades específicas al sistema. También se incluyen componentes de tipo **«archivo»**, como *animator.ini*, que suele corresponder a un archivo de configuración; de tipo **«documento»**, como *animator.hlp*, que probablemente contiene ayuda o documentación para el usuario; y de tipo **«tabla»**, como *shapes.tbl*, que representa datos estructurados utilizados por alguna parte del sistema.

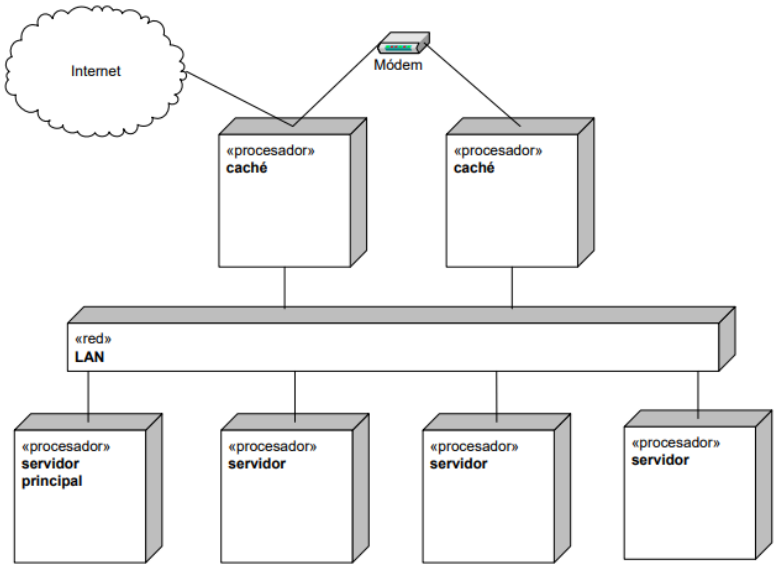
Además de las dependencias directas del ejecutable, el diagrama muestra **dependencias encadenadas entre componentes**. Por ejemplo, *render.dll* depende de *raytrace.dll*.

Diagrama de Despliegue

Un **nodo** es un elemento físico que existe en tiempo de ejecución y supone un recurso computacional, representan hardware sobre el que se despliegan los **componentes** para que se ejecute el sistema.

Nota: los **componentes** y **nodos** pertenecen al mundo **físico**.

El diagrama muestra las relaciones entre componentes software y hardware durante la ejecución. Su propósito es mostrar la **configuración** de los **nodos** que participan en la ejecución y de los **componentes** que residen en ellos, y modelan la **topología** del hardware sobre el que se ejecuta el sistema. Ayuda en decisiones de distribución, rendimiento y escalabilidad.



En la parte superior del diagrama aparece **Internet**, representado como una nube, lo que indica una **red externa** desde la cual llegan las peticiones al sistema. Esta conexión se realiza a través de un **módem**, que actúa como dispositivo intermediario permitiendo la comunicación entre la red externa y la infraestructura interna.

A continuación, se observan dos nodos etiquetados como **«procesador» caché**. Estos representan servidores de caché cuya función es almacenar temporalmente datos o respuestas frecuentes. Ambos están conectados al módem y, por tanto, reciben las solicitudes provenientes de Internet antes de que estas se propaguen al resto del sistema.

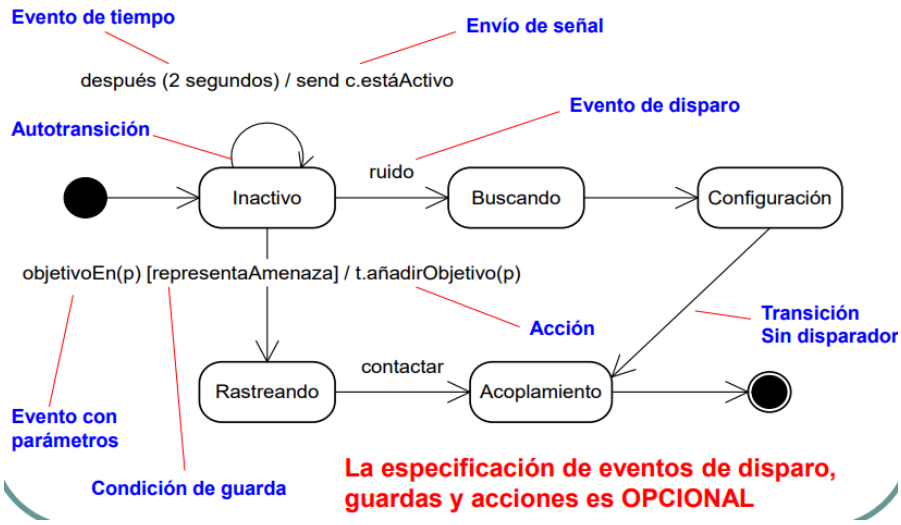
Debajo de estos nodos se encuentra una **red local («red» LAN)**. Esta LAN actúa como el núcleo de la infraestructura interna, conectando todos los servidores del sistema y permitiendo la comunicación entre ellos. En la parte inferior del diagrama aparecen varios nodos etiquetados como **«procesador» servidor**. Las líneas que conectan los distintos elementos representan **enlaces de comunicación**, indicando cómo fluye la información a través del sistema.

Nota: Pueden aparecer diagramas combinados (ej: de despliegue + de componentes, véase diapositiva 14)

Diagrama de Estados

Un **diagrama de estados** muestra una máquina de estados, destacando el flujo de control entre esos estados (cómo cambia el estado de un objeto a lo largo del tiempo en respuesta a eventos). Su propósito es modelar el **comportamiento** de un único **objeto** a través de varios casos de uso. Su contenido trata de estados simples y compuestos, transacciones, eventos y acciones.

Nota: Puede haber **superestados** que agrupan estados similares.



Elementos:

- **Evento de tiempo**: evento que ocurre después de cierto intervalo.
- **Envío de señal**: acción de enviar mensaje o señal a otro objeto o componente.
- **Evento de disparo**: evento externo que provoca una transición entre estados (define como tiene que actuar el sistema y a que estado cambiar).

- **Autotransmisión**: transmisión que empieza y termina en el mismo estado.
- **Evento con parámetros**: evento que incluye datos asociados (para pasar información).
- **Condición de guarda**: condición booleana que dee cumplirse para que ocurra la transición ([condición]).
- **Acción**: operación que se ejecuta cuando ocurre transición, para modificar el estado interno del sistema.
- **Transición sin disparador**: transición que ocurre automáticamente, sin evento explícito (flecha sin texto).

Todo comienza en el estado inicial (círculo negro), desde donde el sistema entra en el estado **Inactivo**. En este estado, el sistema no realiza acciones externas relevantes, pero puede reaccionar ante eventos o condiciones. Mientras está **Inactivo**, pueden ocurrir varias cosas.

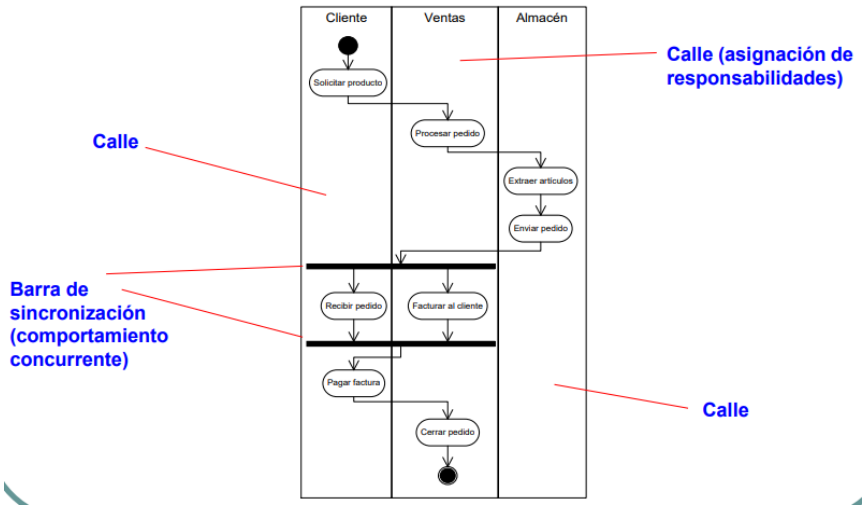
- Existe una **autotransición**, el sistema puede evaluarse o actualizarse internamente sin cambiar de estado.
- Si se detecta un evento como *ruido*, el sistema cambia al estado **Buscando**.
- Si se cumple una condición específica (por ejemplo, detectar un objetivo que representa una amenaza), el sistema pasa a **Rastreando**, ejecutando además una acción asociada (añadir ese objetivo a una lista interna).

En el estado **Buscando**, el sistema puede evolucionar hacia **Configuración**. Desde ahí, sin necesidad de un evento explícito, puede pasar directamente a **Acoplamiento** (transición automática basada en la lógica interna).

En paralelo, si el sistema ya está en **Rastreando**, puede pasar a **Acoplamiento** mediante el evento *contactar*. Finalmente, el sistema llega al estado final (círculo concéntrico), indicando que el proceso ha terminado.

Diagrama de Actividades

Es un **caso especial** de diagrama de **estados** donde la mayoría de las transiciones se disparan debido a la **finalización** de **actividad** en el estado origen. Se usan para modelar escenarios, algoritmos y **flujos** de **trabajo** (workflows) o procesos del sistema. Además, fomentan la búsqueda de actividades paralelas.



Elementos:

- **Calle**: es una división vertical (u horizontal) del diagrama que representa un actor, rol o departamento. Quién es responsable de la actividad.
- **Barra de sincronización**: elemento que permite dividir o unir flujos paralelos.
    - **Paralelismo (dividir)**: iniciar varias actividades en paralelo.
    - **Sincronización (unir)**: esperar a que todas las actividades paralelas terminen.

El flujo comienza en la calle del **Cliente**, donde aparece el nodo inicial (círculo negro). Desde ahí, el cliente realiza la acción **"Solicitar producto"**, iniciando todo el proceso. Esta acción genera una transición hacia la calle de **Ventas**, indicando que la responsabilidad pasa a ese departamento. En **Ventas**, se ejecuta la actividad **"Procesar pedido"**, donde se gestiona la solicitud recibida.

A continuación, el flujo pasa a la calle de **Almacén**, lo que indica que ahora este departamento toma el control. En este punto, el almacén realiza dos acciones secuenciales: **"Extraer artículos"** (preparar los productos) y luego **"Enviar pedido"** (despacharlos al cliente). Una vez enviado el pedido, el flujo vuelve a una zona donde ocurre algo importante: aparece una **barra de sincronización**, que divide el proceso en dos caminos paralelos. Esto significa que se inician simultáneamente dos actividades en diferentes carriles:

- En **Cliente**: **"Recibir pedido"** y en **Ventas**: **"Facturar al cliente"**

Después, aparece otra barra de sincronización que vuelve a unir ambos caminos. Esto implica que el proceso no puede continuar hasta que ambas actividades paralelas hayan finalizado. A continuación, el cliente realiza la acción **"Pagar factura"**, lo que representa la finalización de su parte en el proceso económico. Luego, el flujo pasa nuevamente al carril de **Ventas**, donde se ejecuta la acción final **"Cerrar pedido"**. Finalmente, el proceso concluye en el **nodo final** (círculo con borde), que simboliza el fin del flujo de actividades.